

CS-317 Operating Systems



Lecture 7



Threads

Book 1 Chapter 4

Processes and threads



What is a thread? How is it different from the process?

The status of a process in an OS



The process is the unit of
resource ownership.



The process is the unit of
scheduling or dispatch.

Program code of a process

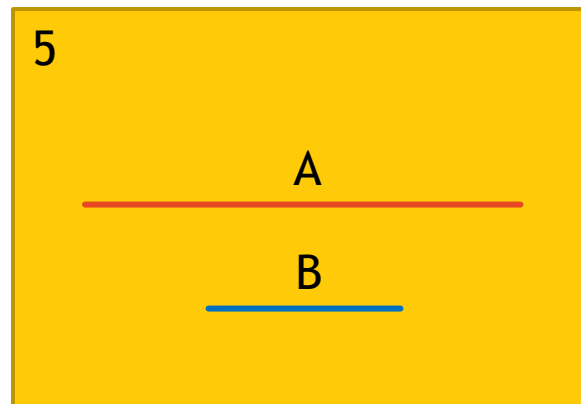
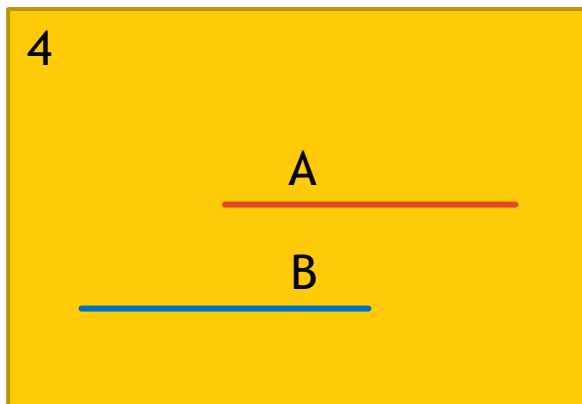
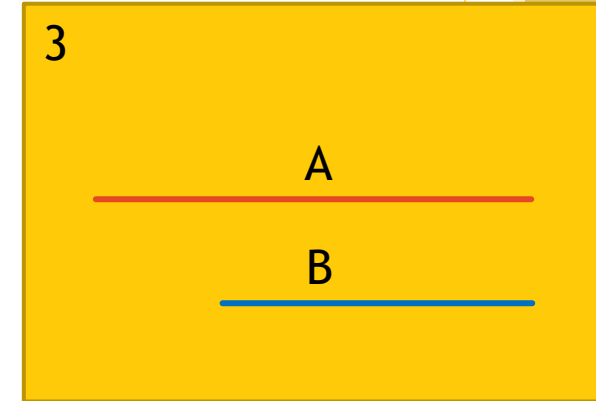
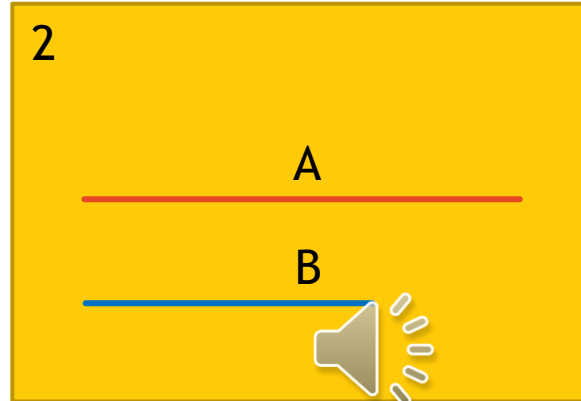
```
def func1(val1):  
    .  
    .  
    .  
    return a  
  
def func2(val2):  
    .  
    .  
    .  
    return b
```

- ▶ A program has two functions.
- ▶ We do not know if both functions use the same data as input or not.
- ▶ We know that the two functions have no data dependency.
- ▶ Conventional compilers run the functions one after the other.
- ▶ Concurrent compilers can run
 - ▶ func2 after func1 (sequential execution)
 - ▶ func1 with func2 (parallel execution)
- ▶ There are two flows of execution in this process.
- ▶ There are two paths of execution in this process.

There are two threads in this process.

Concurrency

SIMULTANEITY



Two events are concurrent when their life spans overlap, completely or partially.

- ▶ It refers to the ability of an operating system to support multiple, concurrent paths of execution within a single process.
- ▶ The traditional OS, which recognizes a single flow of execution in a process, is called a single-threaded operating system.
- ▶ A multi-threaded OS schedules not processes, but the threads that exist within processes.

Multi-threading



Point to note!

In a system that supports threads, the process remains the unit of resource ownership and the unit of protection, but not the unit of dispatch.



I need
file XYZ!



THUD!!!

What was
that?



Did you hear that
kitty? Someone asked
for something again.

Here you go, whoever you are.
The location service only
identifies your building.
Dropping request on the roof!!

THE OS
RESPONDING TO
A RESOURCE
REQUEST FROM A
THREAD INSIDE A
PROCESS

Process-thread arrangements and OS types

SINGLE-THREADED APPROACHES



MULTI-THREADED APPROACHES



Thread attributes

these are thread properties

- ▶ A thread execution state,
- ▶ A saved thread context while not running,
- ▶ An execution stack,
- ▶ Some per-thread static storage for local variables,
- ▶ Access to memory and resources of its process, shared with all the other threads of this process.



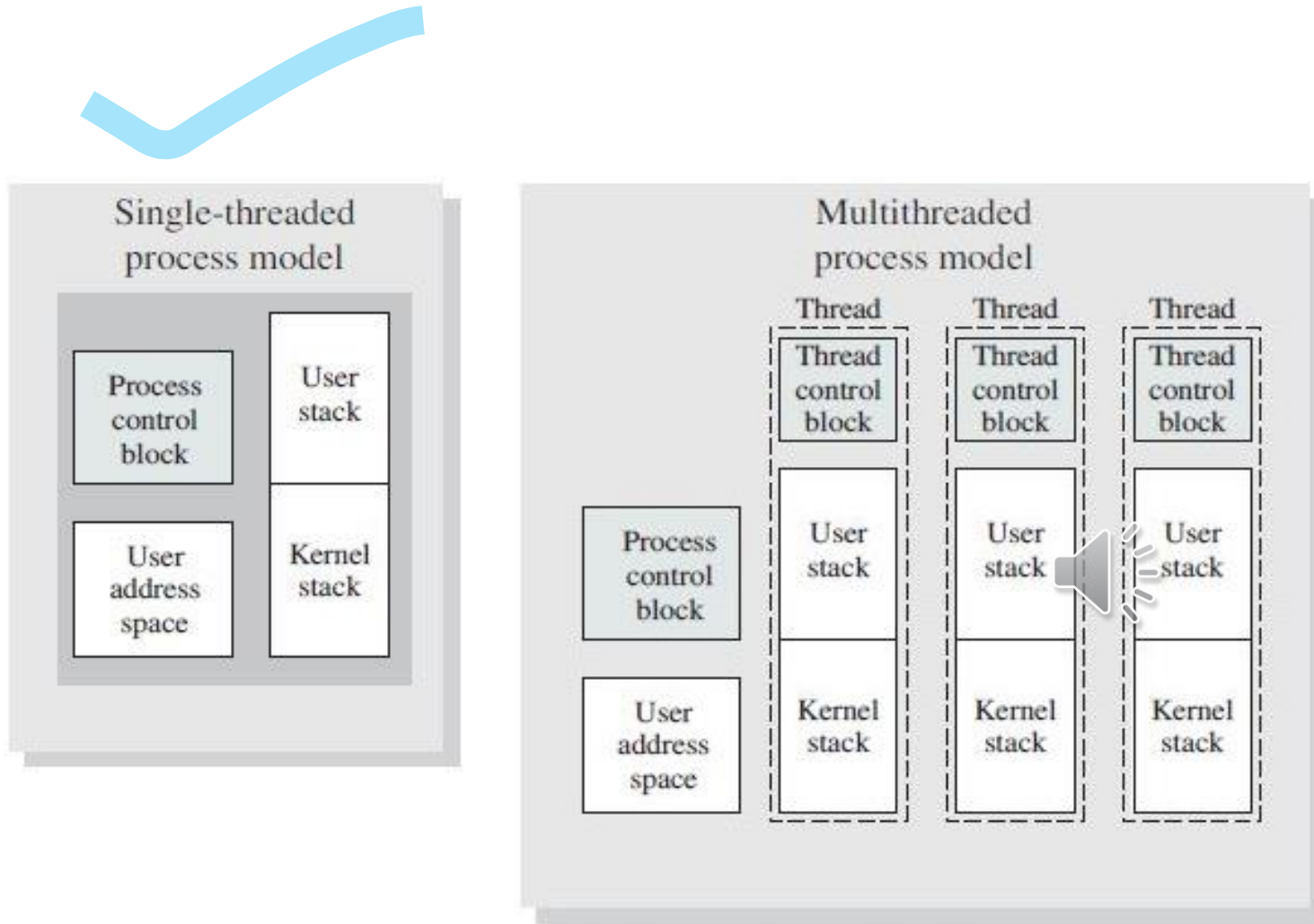


Figure 4.2 Single-Threaded and Multithreaded Process Models

The
manifestation
of threads
within a
process

What did we learn so far?

- ▶ Threads of a process share the state and resources of that process.
- ▶ They reside in the same address space and have access to the same data.
- ▶ If one thread alters a data piece in the address space of the process, all other threads can see this change.
- ▶ If one thread asks for read privileges on a file, and the OS grants a file handle to the process with permissions of read access, all other threads in the process can also read this file.



Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_7. If your roll no is 500, the file will be named 500_lecture_7. Do not add <> signs to the filename.

A thread is the smallest unit of execution within a process.

Each thread has its own program counter, which keeps track of the current instruction it is executing. Multiple threads can exist inside a single process, sharing the same memory and resources. Because each thread runs independently, it can execute different parts of a program at the same time (concurrently). So, the phrase means that a thread behaves like a mini-program running inside a larger program, with its own flow of control.

Task# 1: “One way to view a thread is an independent program counter operating within a process.”

– Book 1, page 160

Explain this statement.

1. **Creation:** It takes far less time to create a new thread in an existing process than the time it takes to create a new process.
2. **Termination:** It takes far less time to terminate a thread in an existing process than the time it takes to terminate a process.

Key benefits of threads



3. **Switching:** It takes far less time to switch between two threads in the same process than the time it takes to switch between two processes.
4. **Communication:** It takes far less time to communicate between two threads in the same process than the time it takes to communicate between two processes.

Key benefits of threads



“ If there is an application or function that should be implemented as a set of related units of execution, it is far more efficient to do so as a collection of threads rather than a collection of separate processes. ”



– Book 1, page 161

- Threads will be made faster.
- They will die faster.
- They will switch faster.
- They will communicate faster.

Pause the video!

Write your answer in a word file previously saved as <my_roll_no>_lecture_7.

Task# 2: How can I be so sure that the size of the TCB is much smaller than the size of the PCB?

Hint: Look at the contents of the PCB. Will all of them be retained in a TCB?

Use of threads on a multiprocessor:

File server

- ▶ As each new file request comes in, a new thread is spawned (created) for the file management program.
- ▶ Because a server will handle many requests, many threads will be created and destroyed in a short period.
- ▶ If the server runs on a multiprocessor computer, then multiple threads within the same process can be executing simultaneously on different processors.
- ▶ Further, because processes or threads in a file server must share file data and therefore coordinate their actions, it is faster to use threads than processes for this coordination.

Use of threads on a uniprocessor:

Foreground and background work

- ▶ In a spreadsheet program, one thread could display menus and read user input, while another thread executes user commands and updates the spreadsheet.
- ▶ This arrangement increases the perceived speed of the application by allowing the program to prompt for the next command before the previous command is complete.

Use of threads on a uniprocessor:

Asynchronous processing

- ▶ Asynchronous elements in the program can be implemented as threads.
- ▶ In a synchronous programming model, things happen one at a time.
- ▶ An asynchronous model allows multiple things to happen at the same time.
- ▶ As a protection against power failure, one can design a word processor to write its RAM buffer to disk once every minute.
- ▶ A thread can be created whose sole job is periodic backup.
- ▶ The main program can respond to the user, without worrying about saving work that has been done so far.



It means adding a safety feature to a word processor so you don't lose your work if the power goes out.

A word processor keeps what you type in RAM (memory), which is temporary. If power fails, everything in RAM is lost. To prevent this, the program is designed to automatically save (write) the contents of RAM to the disk every minute.

Use of threads on a uniprocessor:

Speed of execution

- ▶ A multithreaded process can compute one batch of data while reading the next batch from a device.
- ▶ Even though one thread may be blocked for an I/O operation to read in a batch of data, another thread may be executing, computing result from the batch of data already fetched.

Use of threads on a uniprocessor:

Modular program structure

- ▶ Programs that involve a variety of activities or a variety of input and output may be easier to design and implement using threads.
- ▶ Each activity is implemented in a separate thread.
- ▶ This brings modularity to the programs.
- ▶ Modular code is easy to debug and can be easily reused.



So we conclude that...

Multithreading is beneficial not just when the underlying machine has many processors to distribute the threads to, but even when there is only one processor in the system.



Lessons learned

- ▶ We knew what a process is. Now we know what a thread is.
- ▶ We know the benefits that threads provide us.
- ▶ We looked at scenarios where the use of threads leads to better applications.

Things to do



Read the book.



I will ask you to submit the word file previously saved as <my_roll_no>_lecture_7 in Google classroom.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.